# Market Risk Project Report

ESILV - Market Risk (2025–2026)

**Authors**

Matthieu HANNA GERGUIS & Renaud DE L'EPINE

**Group**

A4-IF3

**Module Coordinator**

Matthieu GARCIN

**Professor**

Nicolas PESCI

December 29, 2025

Repository: `https://github.com/MatthieuHannaGerguis/MarketRisk_Project_Hanna_Gerguis_DeLEpine`

# Contents

# Project Overview

This report follows the same flow as our notebook: we go question by question, keep the writing simple, and focus on the essentials: what we did, what we found, and how to read the results.

The notebook is meant to stand on its own. It is well commented, clearly written, and (we hope) really complete. So if you want to double-check a step, rerun the code, or see extra outputs and plots, the notebook should basically explain itself.
We tried to keep the same level of care in this report too: clean layout, clear results, and straightforward interpretations, while staying consistent with the notebook.
We also used Git and GitHub during the project to keep everything tidy and track changes. The repository is here: `https://github.com/MatthieuHannaGerguis/MarketRisk_Project_Hanna_Gerguis_DeLEpine`

We keep it simple: at the start of each question, we quickly say what data we are using and, if needed, what we did to prepare it. That way, you always know right away what we are working with, without having to guess or scroll back.

We also make our conventions clear whenever it matters, so there is no confusion. For example, we always say how returns are defined, what we call a "loss", which confidence level we use, and, when needed, the sign convention (like $\varepsilon \in \{-1, +1\}$ for trade direction).

# Question A - Historical VaR (Non-parametric)

The goal of Question A is to measure the market risk of the Natixis stock with a historical (non-parametric) Value-at-Risk approach. So we do not assume a parametric distribution for returns: we will use the empirical information in the data.
We work with the file `Natixis_stock.txt`. It contains daily prices (date + price). In Question A, the VaR is computed on a one-day horizon, and we use the risk level $\alpha = 5\%$ (so a 95% confidence level).
We will use two time windows in the project: (i) 2015–2016 will be the in-sample period (to estimate the VaR), (ii) 2017–2018 will be the out-of-sample period (to validate later, in Question A.b).

**Notes:** From now on, we call a negative return a loss. So a VaR at level 5% will typically be a negative number (left tail).

## I. Data preparation and return computation (before Question A.a)

We start by loading the Natixis price data and computing daily returns. This is just basic preprocessing, but it matters a lot: if dates are not parsed correctly or if prices are not numeric, everything later becomes wrong.
Also, we will reuse the exact same dataset and the exact same return series later (Questions B and C). So we do the cleaning and the return computation once, and we keep it consistent for the whole report.
**Notes:** The prices are read from a `.txt` file, so the "Value" column is not automatically a float. In the raw file, decimals use a comma, so we must replace commas by dots before converting to float.

```
[90]: import numpy as np
      import pandas as pd
      import matplotlib.pyplot as plt

      #data = pd.read_csv("../data/Natixis_stock.txt", delimiter='\t',
                         #header=None, parse_dates=[0]) #we will mainly use main dataframe to analyze data in our whole project
      data = pd.read_csv("../data/Natixis_stock.txt", delimiter='\t',
                         header=None, parse_dates=[0])
      data.columns = ['Date', 'Value'] # rename columns
      data['Date'] = pd.to_datetime(data['Date'], format="%d/%m/%Y")

      data['Value'] = data['Value'].str.replace(',', '.').astype(float) #retype float number as we read a txt
      data = data.sort_values('Date') # let's sort by date to calculate returns

      def Returns(data): #function to go to the next row in the same column(we didn't use pct_change() a pandas functiob)
          data['Returns'] = (data['Value'] - data['Value'].shift(1)) / data['Value'].shift(1)
          return data

      data = Returns(data)
      print(data)

      #the first value of returns is NaN which is logical so we decided to erase the row from our dataset
      data = data.dropna()
      print(data)


      alpha = 0.05 # tail probability / risk level (=>95% confidence level)
      value_portfolio = 100000 #value portfolio is the actual value of the portfolio
```

Figure 1: Python code used to load Natixis prices, clean the columns, and compute daily returns.

Now we explain quickly what the code does.
We load the text file and we rename the two columns into `Date` and `Value`. Then we convert `Date` into a proper date format, because we will filter the sample by years later. We also make sure `Value` is numeric (float), since the raw file can contain commas as decimal separators. After that, we sort the dataframe by `Date` to keep the correct chronological order.
Finally, we compute simple daily returns using the standard formula:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}.$$

```
           Date  Value   Returns
0    2015-01-02  5.621       NaN
1    2015-01-05  5.424 -0.035047
2    2015-01-06  5.329 -0.017515
3    2015-01-07  5.224 -0.019704
4    2015-01-08  5.453  0.043836
...         ...    ...       ...
1018 2018-12-21  4.045 -0.001481
1019 2018-12-24  4.010 -0.008653
1020 2018-12-27  3.938 -0.017955
1021 2018-12-28  4.088  0.038090
1022 2018-12-31  4.119  0.007583

[1023 rows x 3 columns]
           Date  Value   Returns
1    2015-01-05  5.424 -0.035047
2    2015-01-06  5.329 -0.017515
3    2015-01-07  5.224 -0.019704
4    2015-01-08  5.453  0.043836
5    2015-01-09  5.340 -0.020723
...         ...    ...       ...
1018 2018-12-21  4.045 -0.001481
1019 2018-12-24  4.010 -0.008653
1020 2018-12-27  3.938 -0.017955
1021 2018-12-28  4.088  0.038090
1022 2018-12-31  4.119  0.007583

[1022 rows x 3 columns]
```

Figure 2: Output check: the dataset contains `Date`, `Value` and the computed `Returns`.

At this stage, the dataset is ready. We have clean prices, clean dates, and one return per trading day. We also define $\alpha = 0.05$ and a portfolio value (here 100,000) because later we will express risk in monetary terms if needed.
Bonus: we plot the Natixis stock price evolution, just to see the overall dynamics and make sure the time series looks consistent.

```
[108]: plt.figure(figsize=(10, 5)) # simple visualization of the Natixis stock price over time
       plt.plot(data["Date"], data["Value"])
       plt.xlabel("Date")
       plt.ylabel("Stock price")
       plt.title("Natixis stock price evolution")
       plt.show()
```

Figure 3: Code used to plot Natixis stock price evolution.

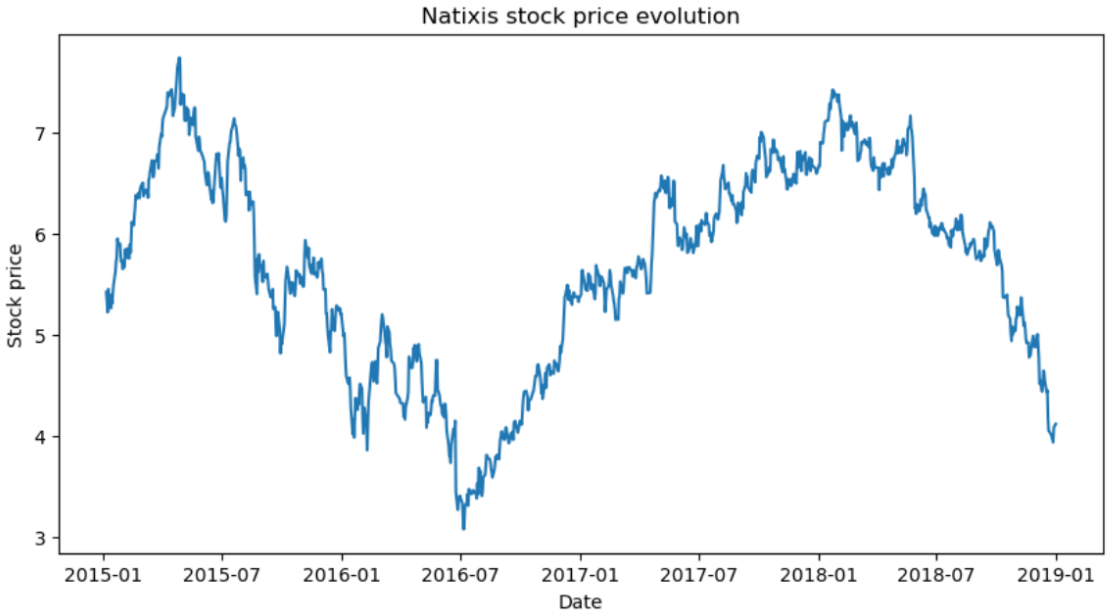



Figure 4: Natixis stock price evolution (visual check of the raw time series).

**Notes:** This plot is not used directly to compute the VaR. It is just a sanity check: it helps us confirm that the dates are in the right order and that the series behaves like a real price series.

## II. Historical VaR estimation (Question A.a)

In this part, we estimate a one-day historical VaR for Natixis returns. We keep the same settings as in Part I: we work with daily simple returns, we use $\alpha = 0.05$ (so a 95% confidence level), and we take a portfolio value of 100,000 to express VaR in euros too.
We follow the notebook logic: first we isolate the in-sample period (January 2015 to December 2016) that we will use to *estimate* the VaR. (Then, in Question A.b, we will use January 2017 to December 2018 to validate it out-of-sample.)

```
[9]: # on our sorted dataset we select the time periods required to estimate and validate the historical VaR.
data_date_sort_2015 = data[(data["Date"] >= '2015-01-01') & (data["Date"] <= '2016-12-31')]
data_date_sort_2017 = data[(data["Date"] >= '2017-01-01') & (data["Date"] <= '2018-12-31')]

#Once the selection is done now we can sort returns !
data_date_sort_2015 = data_date_sort_2015.sort_values("Returns") # using build in sort function of pandas
data_date_sort_2017 = data_date_sort_2017.sort_values("Returns")

print(data_date_sort_2015) # quick check up
```

Figure 5: Selecting the estimation window (2015–2016) and the validation window (2017–2018), then sorting by returns.

```
           Date  Value   Returns
378 2016-06-24  3.439 -0.171325
345 2016-05-10  4.083 -0.069083
358 2016-05-27  4.460 -0.061448
405 2016-08-02  3.422 -0.060664
277 2016-02-02  4.208 -0.058613
..         ...    ...       ...
283 2016-02-10  4.101  0.062435
305 2016-03-11  5.083  0.063389
164 2015-08-25  5.755  0.064755
403 2016-07-29  3.685  0.075912
326 2016-04-13  4.785  0.078431

[512 rows x 3 columns]
```

Figure 6: Quick check: 2015–2016 returns sorted from worst to best.

**Course reference:** We recall the definition of the historical Value at Risk as introduced during the lectures (see slide on Value at Risk, page 68).

**Definition**

*Let $X$ be a random variable depicting a gain. Then, the value at risk with confidence $\alpha$ is:*

$$VaR_\alpha(X) = -F_X^{-1}(1-\alpha).$$

*Alternatively, if $Y$ is a loss ($Y = -X$), $VaR_\alpha(Y) = F_Y^{-1}(\alpha)$.*

Figure 7: Course slide (page 68): Definition of the historical Value at Risk.

Now we compute the historical VaR at level $\alpha$. In the notebook, we first use the empirical quantile (standard historical VaR). So we simply take the $\alpha$-quantile of the observed returns.

If $r$ is the daily return, the VaR threshold on returns is:

$$\mathrm{VaR}_{\alpha}^{(\mathrm{ret})} = q_{\alpha}(r),$$

where $q_{\alpha}(r)$ is the empirical $\alpha$-quantile.

**Convention:** returns can be negative, so $\mathrm{VaR}_{\alpha}^{(\mathrm{ret})}$ is usually negative (left-tail threshold). If we want a positive loss number (in % or in euros), we report $-\mathrm{VaR}$.

```
[91]: # Historical VaR estimation using the empirical quantile
      var_historical = np.quantile(data_date_sort_2015["Returns"], alpha)

      var_historical_value = - var_historical * value_portfolio # we have VaR expressed in monetary value
      var_historical, var_historical_value

[91]: (np.float64(-0.03777913198405718), np.float64(3777.9131984057176))
```

Figure 8: Empirical (historical simulation) VaR: return quantile and monetary VaR.

With $\alpha = 0.05$, the historical VaR on returns is about $-0.03778$. So the threshold is around -3.78% in one day. If we take a portfolio value of 100,000, this gives a monetary VaR of about 3,777.91 euros.

What does it mean? Looking at 2015-2016, only 5% of the days have a return worse than -3.78%. So, with this historical definition, a "bad" 5% day means a loss of at least about 3,778 euros.

**Convention:** the VaR on returns is negative because we look at the left tail. When we want a positive risk number (a loss), we just take $-\mathrm{VaR}$, and in euros we multiply by the portfolio value.

Now we do the same idea, but with a smoother distribution. This is what the instructions ask for: a non-parametric VaR using a kernel approach. Instead of using the empirical distribution (which is step-like), we smooth the return distribution with a KDE, and then we take the same 5% quantile.

This method is exactly the one shown in the course slides (VaR estimation slide, page 69), and it is also the reference used in the notebook.

The kernel we use is the biweight kernel:

$$K(u) = \begin{cases} \frac{15}{16}(1-u^2)^2 & \text{if } |u| \le 1, \\ 0 & \text{otherwise.} \end{cases}$$

The kernel is also given in the project statement, and the notebook shows it with a "wave" shape.

With this kernel, the KDE at point $x$ is:

$$\widehat{f}(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - r_i}{h}\right),$$

where $r_i$ are the observed returns (2015-2016 here) and $h$ is the bandwidth.

In the code, the bandwidth $h$ is chosen from the data (it uses the return standard deviation and the sample size), so it automatically adapts when the dataset size changes.

[11]:
```python
def kernel(u): # we define the biweight kernel used in the non-parametric estimation
    if abs(u) <= 1:
        return (15/16) * (1 - u**2)**2
    else:
        return 0

def kernel_density(data, x): # Kernel density estimation function where x is the point where the density is evaluated
    # bandwidth chosen
    h = ((4 * data['Returns'].std()) / (3 * len(data['Returns'])))**0.2
    n = len(data['Returns'])
    kernel_sum = 0

    # we sum the contribution of each observation to the kernel density
    for r in data['Returns']:
        kernel_sum += kernel((x - r) / h)
    return kernel_sum / (n * h)     # final kernel density value

def VaR_Kernel(data, alpha): # function to compute the non-parametric VaR using the kernel distribution
    probabilities = []
    for r in data['Returns']:
        probabilities.append(kernel_density(data, r))

    total = sum(probabilities)  # normalization to ensure probabilities sum to one
    if total != 0:
        probabilities = [p / total for p in probabilities]
    sorted_returns = sorted(data['Returns'])     # computation of the empirical quantile based on cumulative probabilities
    cumulative_prob = 0

    for i, p in enumerate(probabilities):
        cumulative_prob += p
        if cumulative_prob >= alpha:
            return sorted_returns[i]

Value_at_Risk = VaR_Kernel(data_date_sort_2015, 0.05)
print(f"Value at Risk of Kernel : {Value_at_Risk}")
```

```
Value at Risk of Kernel : -0.03478608556577359
```

Figure 9: Kernel-based non-parametric VaR (biweight KDE + extraction of the 5% quantile).

The kernel-based VaR (return threshold) is:

$$\mathrm{VaR}^{(\mathrm{ret})}_{0.05,\mathrm{kernel}} \approx -0.03479,$$

so around -3.48% in one day.

If we convert to euros (same portfolio value 100,000), the loss is about:

$$-\mathrm{VaR}^{(\mathrm{ret})}_{0.05,\mathrm{kernel}} \times 100{,}000 \approx 3{,}478.61 \text{ euros}.$$

This is positive because the return VaR is a negative left-tail threshold.

Finally, the notebook plots the KDE with the return histogram and the VaR cutoff (vertical line). This is just a quick visual check to see where the 5% tail starts.

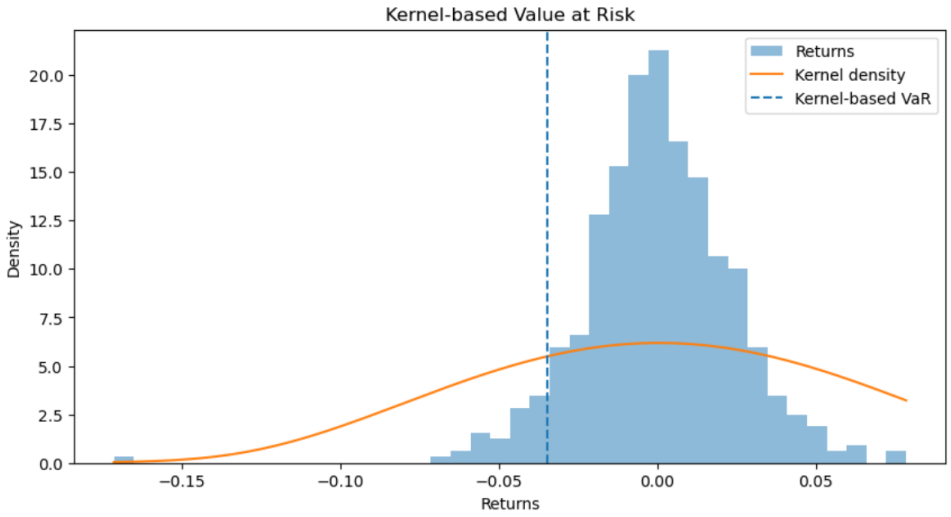



Figure 10: Kernel density estimate of returns (2015–2016) and the 5% VaR threshold.

**Notes:** Here the kernel VaR is slightly less extreme than the pure empirical VaR. That can happen because KDE smooths the distribution, so the left tail is a bit "less step-like". In Question A.b, we will check out-of-sample (2017–2018) whether this VaR level is still realistic.

## III. Out-of-sample validation (2017–2018) (Question A.b)

We now check if the non-parametric VaR estimated on 2015–2016 behaves well on new data. So we take the out-of-sample period 2017–2018 and we count how often returns fall below the VaR threshold.

**Convention:** here, an "exceedance" means a return that falls below the VaR threshold, because the VaR on returns is a left-tail cutoff. So we count days where $r_t < \text{VaR}_\alpha$.

In the notebook, this is done with a simple loop: we extract the 2017–2018 returns, we count how many are below `Value_at_Risk` (the KDE VaR from Part II), and we compute the proportion.

[13]:
```
# We now look at how many returns exceed the VaR threshold on the 2017-2018 period

data_2017_2018 = data[
    (data['Date'] >= '2017-01-01') &
    (data['Date'] <= '2018-12-31') # out-of-sample period (2017-2018)
]
returns_2017_2018 = data_2017_2018['Returns']

exceedances = 0

for r in returns_2017_2018:
    if r < Value_at_Risk:
        exceedances += 1 #  count how many times returns fall below the estimated VaR
exceedance_ratio = exceedances / len(returns_2017_2018) #  proportion of exceedances

print("With non-parametric VaR")
print(exceedances, "returns out of", len(returns_2017_2018), "exceed the VaR threshold   ")
print("This corresponds to", round(exceedance_ratio * 100, 2), "% for a risk level of 5%")
```

```
With non-parametric VaR
9 returns out of 510 exceed the VaR threshold
This corresponds to 1.76 % for a risk level of 5%
```

Figure 11: Out-of-sample check (2017–2018): count how many returns fall below the KDE VaR threshold.

In our case, we get: 9 exceedances out of 510 returns, so an exceedance frequency of 1.76%.

We just compare this to the nominal level $\alpha = 5\%$. Since 1.76% is below 5%, we observe fewer exceedances than expected for a nominal 5% VaR. So, on this 2017–2018 period, the kernel-based VaR looks conservative.

**Notes:** this kind of historical / kernel VaR implicitly assumes that the past return dynamics (2015–2016 here) stay representative of the future (2017–2018). In real markets this may fail, especially if volatility regimes change, so this validation step is important.

# Question B - Expected Shortfall

In this question, we compute the Expected Shortfall associated with the non-parametric VaR estimated in Question A.

We reuse the same return series as in Question A and the same non-parametric VaR threshold (`Value_at_Risk`). So we do not re-estimate VaR here. We only compute ES using the VaR level already obtained before.

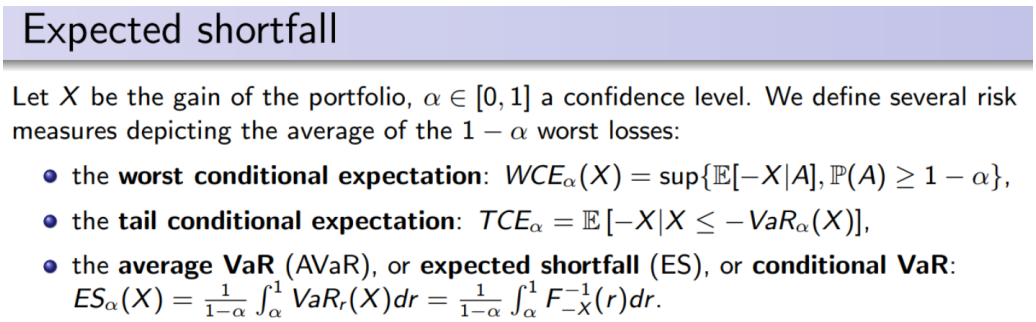
The idea is simple. VaR gives a cutoff (a quantile of the return distribution). Expected Shortfall goes one step further: it measures the average return when we are in the VaR tail, meaning when returns fall below the VaR threshold. So we simply average the returns that fall below the VaR threshold.

**Sign convention:** here we work with returns (not losses). So the left-tail VaR and the ES are negative numbers. In practice, we select the tail with $R \leq VaR_\alpha$ (this is what the code does with `Returns <= Value_at_Risk`), and then ES is the average of these tail returns, so it is also negative. If we want to report a positive loss number, we simply use $-$VaR and $-$ES (and in euros we multiply by the portfolio value).

**Course reference:** Expected Shortfall (also called AVaR or conditional VaR) is introduced in the lectures as the average loss given we are in the VaR tail, i.e. when returns fall below the VaR threshold (see slide "Expected shortfall", page 75). In the lectures, Expected Shortfall is also presented as a spectral (distortion) risk measure. For a continuous distribution, it can be written as:

$$ES_\alpha = \mathbb{E}[R \mid R \leq VaR_\alpha].$$

This is the definition we use here (same as in the notebook).



Expected shortfall

Let $X$ be the gain of the portfolio, $\alpha \in [0,1]$ a confidence level. We define several risk measures depicting the average of the $1-\alpha$ worst losses:

- the **worst conditional expectation**: $WCE_\alpha(X) = \sup\{\mathbb{E}[-X|A], \mathbb{P}(A) \geq 1-\alpha\}$,
- the **tail conditional expectation**: $TCE_\alpha = \mathbb{E}[-X|X \leq -VaR_\alpha(X)]$,
- the **average VaR** (AVaR), or **expected shortfall** (ES), or **conditional VaR**: $ES_\alpha(X) = \frac{1}{1-\alpha}\int_\alpha^1 VaR_r(X)dr = \frac{1}{1-\alpha}\int_\alpha^1 F^{-1}_{-X}(r)dr$.

Figure 12: Course slide (page 75): Expected Shortfall definition (ES / AVaR / conditional VaR), as referenced in the notebook.

**Computation of the Expected Shortfall:**
In practice, we take all in-sample returns (2015-2016) that are below (or equal to) the non-parametric VaR, and we compute their mean. This is exactly what the notebook does.

```
[113]: # select the returns that are below (or equal to) the VaR threshold (worst tail)
rdt_over_VaR = data_date_sort_2015["Returns"][data_date_sort_2015["Returns"] <= Value_at_Risk]
# We compute ES empirically:we average the returns in the left tail (R <= VaR_alpha),
# which matches ES_alpha = E[R|R <= VaR_alpha].
ES = np.mean(rdt_over_VaR)

print("Non parametric VaR:", Value_at_Risk)
print("Expected Shortfall (non-parametric):", round(ES, 5))
```

```
Non parametric VaR: -0.03478608556577359
Expected Shortfall (non-parametric): -0.05004
```

Figure 13: Code: selecting the left tail ($R \leq VaR_\alpha$) and computing ES as the average tail return.

From the notebook output, the non-parametric VaR (95%) is about -3.48% and the ES (95%) is about -5.00%. This makes sense: ES averages the worst returns in the 5% tail, so it tells us what the return looks like on average when we are beyond the VaR threshold.

# Question C - Extreme Value Theory (EVT)

In this question, we focus on the analysis of extreme returns using tools from Extreme Value Theory (EVT). We rely on the Natixis return series and study separately extreme gains and extreme losses in order to better understand tail behavior.

We use the same return series as in Question A, but now we zoom in on the extreme tails with an EVT approach.

**Sign convention:** in this section we work in loss terms. When returns are negative, we define losses $= -$returns, so losses are positive numbers. This means EVT VaR is a positive loss. If we want to compare with the previous sections (VaR on returns), we just take the negative of it.

## I. Data preparation and tail separation (EVT setup, before Question C.a)

To apply Extreme Value Theory, we first split the return distribution into two tails corresponding to extreme gains and extreme losses, which are analyzed separately. Extreme losses are defined as ($L_t = -r_t$) so EVT is applied to the right tail of distribution.

In the notebook, we do it in the simplest possible way: we take positive returns as `gains`, and we convert negative returns into positive numbers for `losses`. Then we sort both series, because later EVT estimators use order statistics (sorted samples).

```
[15]: gains = data['Returns'][data['Returns'] > 0]
      losses = -data['Returns'][data['Returns'] < 0]# converts losses into positive values, thus EVT applies to the right tail.

      gains_sorted = np.sort(gains)
      losses_sorted = np.sort(losses)
```

Figure 14: Tail separation for EVT: gains ($R > 0$) and losses ($L = -R > 0$), then sorting for later EVT estimators.

**Notes:** the key point is the loss transformation. By using $L = -R$ for negative returns, we work with positive losses and we can study extremes on the right tail (large values of $L$). This is why EVT VaR will be positive in the next parts.

## II. Estimation of GEV parameters using Pickands estimator (Question C.a)

In this part, we estimate the tail index of the return distribution using Extreme Value Theory. We rely on the Pickands estimator to characterize the behavior of extreme gains and extreme losses separately.

**Course reference:** We use the Pickands estimator introduced in the EVT lectures to estimate the tail index $\xi$. The notebook follows the formula given in the EVT-based VaR estimation slides (page 180).

*then,* **the Pickands estimator**, *defined by:*

$$\xi^{P}_{k(n),n} = \frac{1}{\log(2)} \log\left(\frac{X_{n-k(n)+1:n} - X_{n-2k(n)+1:n}}{X_{n-2k(n)+1:n} - X_{n-4k(n)+1:n}}\right)$$

Figure 15: Course slide (page 180): Pickands estimator for the tail index $\xi$.

**Course reference:** The sign of $\xi$ tells us the type of extreme value distribution. As recalled in the EVT lectures (same section as in the notebook, page 166): $\xi > 0$ corresponds to a Fréchet distribution (heavy tails), $\xi = 0$ to a Gumbel distribution, and $\xi < 0$ to a Weibull distribution.

**Remark**

*Then:*

- *if $\xi > 0$, the GEV is of Fréchet kind,*
- *if $\xi = 0$, the GEV is of Gumbel kind,*
- *if $\xi < 0$, the GEV is of Weibull kind.*

Figure 16: Course slide (page 166): interpretation of the sign of $\xi$ (Fréchet / Gumbel / Weibull).

Now we apply the Pickands estimator separately to extreme gains and to extreme losses. We use the sorted samples built in Part I: `gains_sorted` and `losses_sorted`.

In the notebook, we implement a small function `pickands_estimator(extreme_values)`. We set $k = \lfloor \log(n) \rfloor$ (rule of thumb from the lectures) to choose how many tail points we use. Then we take the required order statistics and we compute $\hat{\xi}$ using the Pickands formula.

[98]:
```python
def pickands_estimator(extreme_values):

    #Pickands estimator for the tail index, extreme observations sorted in increasing order.
    n = len(extreme_values)

# we set k = int(log(n)) (rule of thumb from the lectures) to choose how many tail points we use.
    k = int(np.log(n))
    x1 = extreme_values[n - k]  #required order statistics
    x2 = extreme_values[n - 2 * k]
    x3 = extreme_values[n - 4 * k]

    # we seek the Pickands estimator according to the theoretical formula
    xi_hat = (1 / np.log(2)) * np.log((x1 - x2) / (x2 - x3))
    return xi_hat

# we estimate the tail index separately for gains and for losses
xi_gains = pickands_estimator(gains_sorted)
xi_losses = pickands_estimator(losses_sorted)
xi_gains, xi_losses
```

[98]: `(np.float64(0.5772338569463286), np.float64(-0.508971577934174))`

Figure 17: Pickands estimator (code) and tail index estimates (output) for gains and losses.

From the notebook, we obtain a positive tail index for extreme gains ($\xi \approx 0.58$) and a negative tail index for extreme losses ($\xi \approx -0.51$). This indicates an asymmetric tail behavior in Natixis returns.

Using the lecture interpretation: $\xi > 0$ for gains suggests heavy-tailed behavior (Fréchet-type extremes), while $\xi < 0$ for losses suggests lighter tails (Weibull-type extremes).

**Notes:** In standard EVT applications, we often assume returns are independent and identically distributed. Also, the Pickands estimator uses only a limited number of extreme observations, so the estimate can be sensitive to sampling variability and to the choice of the intermediate sequence $k$.

Bonus (as in the notebook): we plot the empirical distributions of extreme gains and extreme losses. This helps visualize the difference in tail behavior highlighted by the Pickands estimates.

[114]:
```python
# plot
plt.figure(figsize=(8, 5))

plt.hist(gains_sorted, bins=40, density=True, alpha=0.7,
         label="Extreme gains", edgecolor="black")

plt.hist(losses_sorted, bins=40, density=True, alpha=0.7,
         label="Extreme losses", edgecolor="black")

plt.xlabel("Return magnitude")
plt.ylabel("Density")
plt.title("Empirical distribution of extreme gains and losses")

plt.legend()
plt.grid(True)
plt.show()
```

Figure 18: Code: histogram comparison of extreme gains and extreme losses.

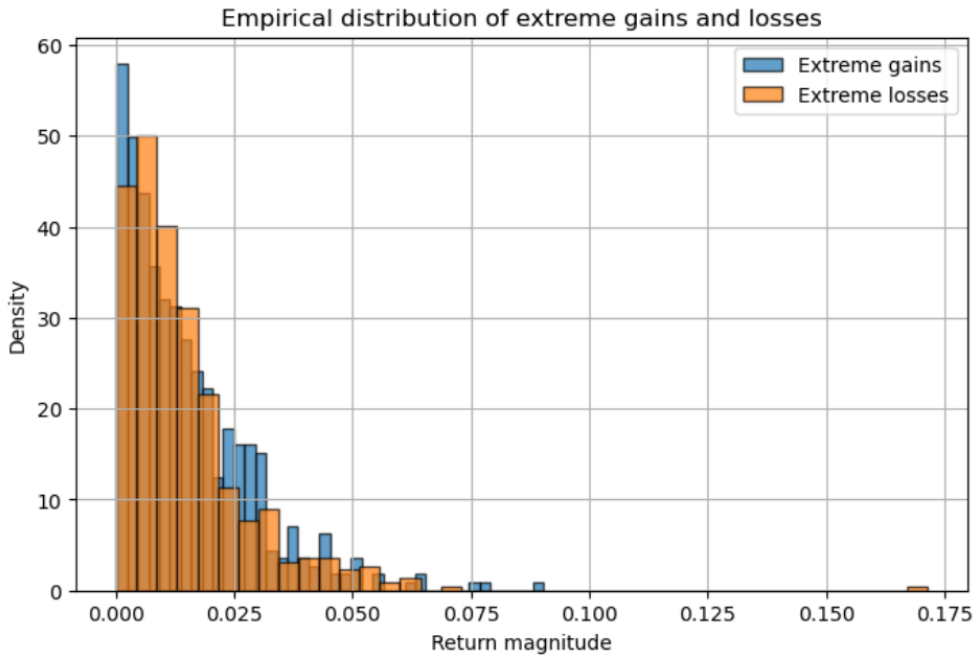



Figure 19: Empirical distribution of extreme gains and extreme losses (visual comparison).

Comment (as in the notebook): the empirical distributions confirm the asymmetrical tail behavior (more pronounced extreme gains than extreme losses).

## III. EVT Value at Risk estimation (Question C.b)

In this part, we use Extreme Value Theory to compute Value at Risk estimates based on the tail behavior characterized before using the Pickands estimator. We focus on the left tail of the return distribution (losses), since Value at Risk is a downside risk measure. The EVT-based VaR is computed for several confidence levels.

The EVT VaR is computed using the tail index estimates obtained in Question C.a. We assume that returns are independent and identically distributed (iid), as in standard EVT applications.

**Course reference:** We use the EVT-based VaR formulation introduced in the lectures where the Value at Risk is expressed using the Pickands estimator of the tail index $\xi$ (see EVT-based VaR estimation slides, page 187).

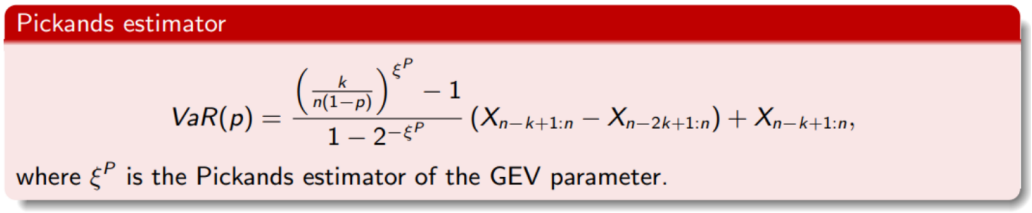



Figure 20: Course slide (page 187): EVT-based VaR formula using the Pickands estimator.

We now compute the EVT Value at Risk for several confidence levels and compare it to the historical VaR obtained in Question A.

**Convention:** Like before, we work with extreme losses expressed in positive terms. So we use $L = -R$ for negative returns. This makes EVT VaR easier to interpret as a loss threshold and avoids sign mistakes in the implementation.

In the notebook, we implement a function `var_evt_pickands(extreme_losses, xi_hat, alpha)`. We keep the same choice of $k = \lfloor \log(n) \rfloor$ to stay consistent with Question C.a. Then we use the required order statistics and the EVT formula from the course to compute VaR for a given confidence level.

[101]:
```python
def var_evt_pickands(extreme_losses, xi_hat, alpha):

    n = len(extreme_losses)
    # we keep the same choice of k to ensure consistency between the two steps
    k = int(np.log(n))
    # tail order statistics required by the EVT formula
    x_k = extreme_losses[n - k]
    x_2k = extreme_losses[n - 2 * k]

    numerator = (k / (n * (1 - alpha))) ** xi_hat - 1
    denominator = 1 - 2 ** (-xi_hat)
    var_evt = (numerator / denominator) * (x_k - x_2k) + x_k
    return var_evt
```

Figure 21: Code: EVT-based VaR computation using Pickands tail index and order statistics.

Then we compute EVT VaR for several confidence levels (as in the notebook: 90%, 95%, and 99%). We apply the formula to `losses_sorted` and we use the estimated tail index for losses (`xi_losses`).

[102]:
```python
confidence_levels = [0.90, 0.95, 0.99] # the  several confidence levels

for alpha in confidence_levels:
    var_alpha = var_evt_pickands(losses_sorted, xi_losses, alpha)
    print(f"EVT-based VaR (Pickands) at {int(alpha*100)}% confidence level: {var_alpha:.5f}")
```

```
EVT-based VaR (Pickands) at 90% confidence level: 0.03141
EVT-based VaR (Pickands) at 95% confidence level: 0.04354
EVT-based VaR (Pickands) at 99% confidence level: 0.05958
```

Figure 22: EVT-based VaR with Pickands: code and printed VaR values for 90%, 95%, 99%.

We can see the VaR going up as we move to higher confidence levels: around 3.1% at 90%, 4.4% at 95%, and about 6.0% at 99%. This is exactly what we expect: higher confidence means we look further into the tail, so losses are rarer but more severe.

**Notes (methodological):** EVT-based VaR relies on extrapolation beyond the observed sample using only the tail behavior of the distribution. So the estimates become more sensitive when the confidence level increases.

Compared to the historical VaR computed in Question A, EVT-based VaR gives more weight to the tail of the loss distribution. As a result, it can lead to higher risk estimates at very high confidence levels. Also, EVT VaR uses only a limited number of extreme observations and can be sensitive to the choice of the sequence $k$ and to sampling variability.

# Question D - Bouchaud Price Impact Model

We now estimate the parameters of Bouchaud's price impact model using the TD4 framework.

For this question, we use the dataset provided for TD4 (`Dataset_TD4.xlsx`), not the Natixis price series used in Questions A–C. The goal here is to focus on market impact and liquidity. So the inputs are different: we work with intraday variables such as price, volume, bid-ask spread, and trade direction (sign).

**Notes:** in this first part, we keep the preprocessing minimal. We mainly rename columns and check the data with a few quick plots. We only do heavier cleaning later, when it becomes necessary for the estimation step (this is the notebook approach).

## I. Data preparation (before Question D)

The objective is to obtain a clean, time-indexed table containing the key variables required by the model: a price variable, traded volume, bid-ask spread $S$, and the trade sign $\varepsilon \in \{-1, +1\}$.

We start by loading the dataset and inspecting its structure. This is just to see what the raw columns look like and to make sure the file is read correctly.

```
[20]: df_td4 = pd.read_excel("../data/Dataset_TD4.xlsx")
      df_td4.head()
```

[20]:

| | transaction date (1=1day=24 hours) | bid-ask spread | volume of the transaction (if known) | Sign of the transaction | Price (before transaction) |
|---|---|---|---|---|---|
| 0 | 0.000202 | 0.1100 | 8.0 | -1 | 100.000 |
| 1 | 0.001070 | 0.1030 | NaN | 1 | 99.984 |
| 2 | 0.001496 | 0.1015 | NaN | -1 | 100.029 |
| 3 | 0.003336 | 0.0920 | NaN | 1 | 99.979 |
| 4 | 0.003952 | 0.1106 | NaN | 1 | 100.060 |

Figure 23: Loading `Dataset_TD4.xlsx` and displaying the first rows.

Then we create a working dataframe and rename the columns. We do this to keep short, clear names that match the model notation used later. So it is easier to read the code and the formulas.

[21]:
```
df_vars = df_td4.copy()

df_vars = df_vars.rename(columns={
    "transaction date (1=1day=24 hours)": "time",
    "Price (before transaction)": "price",
    "volume of the transaction (if known)": "volume",
    "bid-ask spread": "spread",
    "Sign of the transaction": "eps" # copy and rename columns without cleaning the dataset
})

df_vars.head()
```

[21]:

| | time | spread | volume | eps | price |
|---|---|---|---|---|---|
| **0** | 0.000202 | 0.1100 | 8.0 | -1 | 100.000 |
| **1** | 0.001070 | 0.1030 | NaN | 1 | 99.984 |
| **2** | 0.001496 | 0.1015 | NaN | -1 | 100.029 |
| **3** | 0.003336 | 0.0920 | NaN | 1 | 99.979 |
| **4** | 0.003952 | 0.1106 | NaN | 1 | 100.060 |

Figure 24: Creating `df_vars` and renaming columns into `time, spread, volume, eps, price`.

**Convention:** `eps` is the trade sign, with values in $\{-1, +1\}$ (sell vs buy direction). This is the direction variable used in the signed impact model.

Bonus (as in the notebook): before estimating the model, we make two quick plots. These are simple sanity checks to see if the data looks normal.

First, we plot the price evolution over time. This helps us see if the price series behaves normally (no obvious data problem).

[22]:
```
plt.figure(figsize=(8, 4))
plt.plot(df_vars["time"], df_vars["price"])
plt.xlabel("Time")
plt.ylabel("Price")
plt.title("Price evolution over time")
plt.show()
```

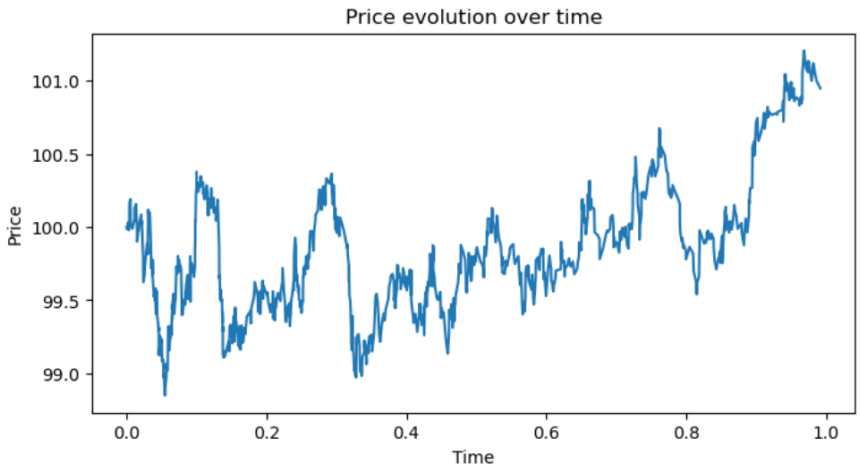



Figure 25: Price evolution over time (quick sanity check).

Second, we look at the distribution of trade signs. This checks if buy and sell directions look roughly balanced, which is important before fitting a signed price impact model.


Figure 26: Distribution of trade signs (buy/sell balance check).

From these plots, we see that trade signs look balanced. This is in line with what we expect before estimating a signed price impact model.

## II. Bouchaud price impact model: definition and parameters

We now introduce Bouchaud's price impact model. We will use it to estimate how trades move prices.

The idea is simple: today's price is the result of many past trades. Each trade pushes the price up or down (because of the trade sign), and this impact slowly fades away over time. So the model keeps both the direction of trades and how long the impact lasts.

**Course reference:** we use the transitory impact model shown in the course (see slide page 133 of the lecture notes).

### Transitory impact model

Backward definition of the price: the price is the sum of past impacts (Bouchaud):

$$p_t = p_{-\infty} + \sum_{s=-\infty}^{t-1} G(t-s)\varepsilon_s S_s V_s^r,$$

Figure 27: Course slide (page 133): Bouchaud transitory impact model.

The slide also explains the model parameters. Here is the same simple interpretation as in the notebook:

$V$**:** overall impact size. If $V$ is larger, a single trade moves the price more.

$r$**:** how impact grows with volume. If $r < 1$, impact is concave (this is commonly observed in real data).

$\gamma$**:** how fast impact decays. A larger $\gamma$ means faster decay (shorter memory). A smaller $\gamma$ means slower decay (longer memory).

Finally, the parameter lambda introduced in the TD4 framework is used to control how past information is weighted over time.

## III. Estimation strategy

We now explain how we estimate the parameters of the transitory impact model. The goal is to recover parameters that control both: (i) the size of the impact, and (ii) how fast this impact fades away.

The general idea is simple: short-term price changes can be explained by the cumulative effect of past signed trades. So we build a predictor from past trades (with a decay over time), and we fit the model by minimizing the difference between observed price changes and predicted price changes.

**Course reference:** this estimation approach follows the TD4 methodology: we choose a parametric form for the impact kernel and we use a regression framework.

We now go step by step.

### 1) Dependent variable: price increments

We first construct the price increments that will be used as the dependent variable. We define:

$$\Delta p_t = p_t - p_{t-1}.$$

This is what the model tries to explain at a short time scale.

[104]:
```
df_est = df_vars.copy()

df_est["delta_price"] = df_est["price"].diff() # we compute price increments from the transaction price

df_est[["time", "price", "delta_price"]].head()
```

[104]:

| | time | price | delta_price |
|---|---|---|---|
| 0 | 0.000202 | 100.000 | NaN |
| 1 | 0.001070 | 99.984 | -0.016 |
| 2 | 0.001496 | 100.029 | 0.045 |
| 3 | 0.003336 | 99.979 | -0.050 |
| 4 | 0.003952 | 100.060 | 0.081 |

Figure 28: Building the dependent variable $\Delta p_t$ using first differences of the transaction price.

### 2) Specification of the impact kernel

To estimate the model, we need a parametric form for the impact decay function. Following TD4, we assume a power-law decay over the lag. We fix a maximum lag $L$ (here $L = 50$), and we define lags:

$$\ell = 1, 2, \ldots, L, \qquad G(\ell) = \ell^{-\gamma}.$$

So $\gamma$ controls the decay speed: larger $\gamma$ means faster decay.

[25]:
```
max_lag = 50
lags = np.arange(1, max_lag + 1) #  lag structure used for the impact kernel

# parametric power law form of the impact kernel
def impact_kernel(lags, gamma):
    return lags ** (-gamma)
```

Figure 29: Power-law impact kernel $G(\ell) = \ell^{-\gamma}$ with a maximum lag $L = 50$.

### 3) Building the regression signal from past trades

For a given pair $(r, \gamma)$, we build a predictor based on past signed trades. We first define the signed trade term:

$$u_t = \varepsilon_t \, S_t \, V_t^{\,r},$$

then we aggregate past values with the kernel:

$$X_t = \sum_{\ell=1}^{L} G(\ell) \, u_{t-\ell} = \sum_{\ell=1}^{L} \ell^{-\gamma} \, \varepsilon_{t-\ell} \, S_{t-\ell} \, V_{t-\ell}^{\,r}.$$

So $r$ controls how impact grows with volume, and $\gamma$ controls how impact decays with time.

In the code, this is implemented with `shift(l)`: we shift the signed term by $\ell$ steps and add it with weight $G(\ell)$. Missing values created by shifts are replaced by 0 (so we do not use trades that do not exist in the past).

### 4) Estimating $V$ by OLS for fixed $(r, \gamma)$

Once we have $X_t$ and $Y_t = \Delta p_t$, we fit a simple linear relation:

$$Y_t \approx V\, X_t.$$

For a fixed $(r, \gamma)$, the best $V$ in least squares has a closed form:

$$\widehat{V} = \frac{\sum_t X_t Y_t}{\sum_t X_t^2}.$$

Then we compute the mean squared error:

$$\mathrm{MSE} = \frac{1}{n} \sum_t \left( Y_t - \widehat{V} X_t \right)^2.$$

We want the parameters that minimize this MSE.

### 5) Grid search on $(r, \gamma)$

We explore a grid of values for $r$ and $\gamma$ and keep the best fit (smallest MSE). In the code, the grids are:

$$r \in \{0.0, 0.1, \ldots, 1.0\}, \qquad \gamma \in \{0.1, 0.2, \ldots, 2.0\}.$$

Before the grid search, we also clean the estimation sample: we keep only rows with non-missing `delta_price`, `eps`, `spread`, and `volume`, and we keep basic checks like $\varepsilon \in \{-1, +1\}$, `spread`$\geq 0$, `volume`$\geq 0$.

[115]:
```python
#we only keep trades with known volume here.
df_fit = df_est.copy()
df_fit = df_fit.dropna(subset=["delta_price", "eps", "spread", "volume"]).reset_index(drop=True)

df_fit = df_fit[df_fit["eps"].isin([-1, 1])]
df_fit = df_fit[(df_fit["spread"] >= 0) & (df_fit["volume"] >= 0)]

max_lag = 50 # We keep a reasonable maximum lag for the kernel
lags = np.arange(1, max_lag + 1)

def impact_kernel_power(lags, gamma):
    # impact decay: G(l) = l^{-gamma}
    return lags ** (-gamma)

def build_regressor(df, r, gamma, max_lag=50):
    """
    We build X_t and we model: delta_price_t ≈ V * X_t
    """
    G = impact_kernel_power(np.arange(1, max_lag + 1), gamma)

    signed_term = df["eps"] * df["spread"] * (df["volume"] ** r)
    X = np.zeros(len(df), dtype=float)
    for l in range(1, max_lag + 1):
        X += G[l - 1] * signed_term.shift(l).fillna(0.0).to_numpy()

    Y = df["delta_price"].to_numpy()
    return X, Y

def estimate_V_ols(X, Y):
    #  OLS
    denom = np.dot(X, X)
    if denom == 0:
        return np.nan
    return np.dot(X, Y) / denom

def mse(Y, Y_hat):
    return np.mean((Y - Y_hat) ** 2)
# grid search
r_grid = np.arange(0.0, 1.01, 0.1) # concavity exponent
gamma_grid = np.arange(0.1, 2.01, 0.1) # decay exponent

best = {"mse": np.inf, "V": None, "r": None, "gamma": None} # structure using the best result

for r in r_grid:
    for gamma in gamma_grid:
        X, Y = build_regressor(df_fit, r=r, gamma=gamma, max_lag=max_lag)
        valid = np.isfinite(X) & np.isfinite(Y)
        Xv, Yv = X[valid], Y[valid] # build regression variables

        V_hat = estimate_V_ols(Xv, Yv) # estimate parameters
        if np.isnan(V_hat):
            continue
        Y_hat = V_hat * Xv
        current_mse = mse(Yv, Y_hat)

        if current_mse < best["mse"]: # jeep the best parameters
            best = {"mse": current_mse, "V": V_hat, "r": r, "gamma": gamma}

best
```

[115]:
```
{'mse': np.float64(0.005377107972098853),
 'V': np.float64(0.001992121301477238),
 'r': np.float64(0.7000000000000001),
 'gamma': np.float64(2.0)}
```

Figure 30: Estimation step: cleaning, kernel-based regressor construction, OLS estimate of $V$, MSE, and grid search over $(r, \gamma)$.

The best fit found is reported as a small summary table (MSE and estimated parameters).

```
[116]: pd.DataFrame([best])
```

[116]:

| | mse | V | r | gamma |
|---|---|---|---|---|
| **0** | 0.005377 | 0.001992 | 0.7 | 2.0 |

Figure 31: Best parameters from the grid search (MSE, $V$, $r$, $\gamma$).

### 6) Result and interpretation

From the printed results, we obtain approximately:

$$\text{MSE} \approx 0.005377, \qquad \widehat{V} \approx 0.001992, \qquad \widehat{r} = 0.7, \qquad \widehat{\gamma} = 2.0.$$

$r$ is below 1, so the impact is concave in volume (this is a common empirical feature). $\gamma$ is high, meaning the impact decays quickly and is mostly temporary (very transitory impact).

This can reflect a liquid market environment, but it can also highlight limitations of the parametric specification or of the chosen lag window.

**Conclusion:** with a simple parametric specification of Bouchaud's transitory impact model, we obtain estimates for the concavity and the decay of price impact. The model captures key features of market impact, but the decay estimate is at the top of the grid ($\gamma = 2.0$), so this looks like a boundary solution. This suggests the true decay might be even faster, or that the model is too restrictive for this dataset.

# Question E - Multiresolution Correlation & Hurst Exponent

In this question, we study FX rates with Haar wavelets and the Hurst exponent. We look at correlations at different time scales and how volatility scales with time.

Here we use a new dataset: `Dataset_TD5.xlsx`. It contains high-frequency FX prices. So before doing the analysis, we first need to clean the data and build proper time series.

## I. Data Description and Preprocessing (Before answering Question E)

The dataset contains three FX pairs: GBPEUR, SEKEUR and CADEUR.

At each time step, we do not have one single price. We have **HIGH** and **LOW**. So we first build one representative price series for each pair, and then we compute returns.

We start by reading the raw Excel file and printing the first lines. This is just to understand how the file is organized.

```
[69]: df_td5 = pd.read_excel("../data/Dataset_TD5.xlsx", header=None)
      df_td5.head(10)
```

[69]:

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |
| 1 | GBPEUR Curncy | NaN | NaN | NaN | SEKEUR Curncy | NaN | NaN | NaN | CADEUR Curncy | NaN | NaN |
| 2 | Date | HIGH | LOW | NaN | Date | HIGH | LOW | NaN | Date | HIGH | LOW |
| 3 | 2016-03-07 08:59:59.990000 | 1.2932 | 1.2917 | NaN | 2016-03-07 08:59:59.990000 | 0.10725 | 0.1072 | NaN | 2016-03-07 08:59:59.990000 | 0.6842 | 0.6829 |
| 4 | 2016-03-07 09:15:00 | 1.294 | 1.293 | NaN | 2016-03-07 09:15:00 | 0.10728 | 0.10717 | NaN | 2016-03-07 09:15:00 | 0.6849 | 0.6841 |
| 5 | 2016-03-07 09:30:00 | 1.2943 | 1.2922 | NaN | 2016-03-07 09:30:00 | 0.10726 | 0.10719 | NaN | 2016-03-07 09:30:00 | 0.6844 | 0.6837 |
| 6 | 2016-03-07 09:45:00 | 1.293 | 1.2913 | NaN | 2016-03-07 09:45:00 | 0.10728 | 0.10721 | NaN | 2016-03-07 09:45:00 | 0.6844 | 0.6839 |
| 7 | 2016-03-07 10:00:00 | 1.2931 | 1.2921 | NaN | 2016-03-07 10:00:00 | 0.10725 | 0.10719 | NaN | 2016-03-07 10:00:00 | 0.684 | 0.6835 |
| 8 | 2016-03-07 10:15:00 | 1.2926 | 1.2921 | NaN | 2016-03-07 10:15:00 | 0.10724 | 0.10718 | NaN | 2016-03-07 10:15:00 | 0.6839 | 0.6836 |
| 9 | 2016-03-07 10:30:00 | 1.293 | 1.2906 | NaN | 2016-03-07 10:30:00 | 0.10722 | 0.1072 | NaN | 2016-03-07 10:30:00 | 0.684 | 0.6834 |

Figure 32: Raw preview of `Dataset_TD5.xlsx`: the three FX pairs are in separate blocks (not a clean table yet).

From this preview, we see the file is not a clean table: the three FX pairs are stored in blocks, and there are empty or unnamed columns. So we reorganize everything into one clean dataframe.

We remove the first 3 rows (they are just labels), we keep only the useful columns, and we rename the columns in a clear way: `Date`, `High`, and `Low` for each FX pair.

We also convert the three date columns into a real datetime format. This is important to keep correct time ordering.

```
[70]: df_clean = df_td5.iloc[3:, [0, 1, 2, 4, 5, 6, 8, 9, 10]] # we remove the first 3 rows of the dataset

df_clean.columns = [
    "Date_GBPEUR", "High_GBPEUR", "Low_GBPEUR",
    "Date_SEKEUR", "High_SEKEUR", "Low_SEKEUR",
    "Date_CADEUR", "High_CADEUR", "Low_CADEUR"
]

df_clean["Date_GBPEUR"] = pd.to_datetime(df_clean["Date_GBPEUR"]) # we rename the column and convert datetime format.
df_clean["Date_SEKEUR"] = pd.to_datetime(df_clean["Date_SEKEUR"])
df_clean["Date_CADEUR"] = pd.to_datetime(df_clean["Date_CADEUR"])

df_clean.shape
df_clean.head()
```

[70]:

| | Date_GBPEUR | High_GBPEUR | Low_GBPEUR | Date_SEKEUR | High_SEKEUR | Low_SEKEUR | Date_CADEUR | High_CADEUR | Low_CADEUR |
|---|---|---|---|---|---|---|---|---|---|
| 3 | 2016-03-07 08:59:59.990 | 1.2932 | 1.2917 | 2016-03-07 08:59:59.990 | 0.10725 | 0.1072 | 2016-03-07 08:59:59.990 | 0.6842 | 0.6829 |
| 4 | 2016-03-07 09:15:00.000 | 1.294 | 1.293 | 2016-03-07 09:15:00.000 | 0.10728 | 0.10717 | 2016-03-07 09:15:00.000 | 0.6849 | 0.6841 |
| 5 | 2016-03-07 09:30:00.000 | 1.2943 | 1.2922 | 2016-03-07 09:30:00.000 | 0.10726 | 0.10719 | 2016-03-07 09:30:00.000 | 0.6844 | 0.6837 |
| 6 | 2016-03-07 09:45:00.000 | 1.293 | 1.2913 | 2016-03-07 09:45:00.000 | 0.10728 | 0.10721 | 2016-03-07 09:45:00.000 | 0.6844 | 0.6839 |
| 7 | 2016-03-07 10:00:00.000 | 1.2931 | 1.2921 | 2016-03-07 10:00:00.000 | 0.10725 | 0.10719 | 2016-03-07 10:00:00.000 | 0.684 | 0.6835 |

Figure 33: Clean table: selected columns, new names, and datetime conversion.

Now we build one price per FX pair. We use the mid-price (average of HIGH and LOW):

$$P_t = \frac{\mathrm{HIGH}_t + \mathrm{LOW}_t}{2}.$$

This gives one price per time step. It also helps reduce microstructure noise compared to using only HIGH or only LOW.

Then we compute simple returns from these average prices:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}.$$

Finally, we drop the missing values created by the return computation.

```
[71]: df_clean["Avg_GBPEUR"] = (df_clean["High_GBPEUR"] + df_clean["Low_GBPEUR"]) / 2
df_clean["Avg_SEKEUR"] = (df_clean["High_SEKEUR"] + df_clean["Low_SEKEUR"]) / 2 # we work with average prices to reduce microstructure noise
df_clean["Avg_CADEUR"] = (df_clean["High_CADEUR"] + df_clean["Low_CADEUR"]) / 2

df_clean[["Avg_GBPEUR", "Avg_SEKEUR", "Avg_CADEUR"]] = \
df_clean[["Avg_GBPEUR", "Avg_SEKEUR", "Avg_CADEUR"]].astype(float)

df_clean["Ret_GBPEUR"] = df_clean["Avg_GBPEUR"].pct_change() # we compute returns from average prices
df_clean["Ret_SEKEUR"] = df_clean["Avg_SEKEUR"].pct_change()
df_clean["Ret_CADEUR"] = df_clean["Avg_CADEUR"].pct_change()

df_clean = df_clean.dropna()

print (df_clean)
```

Figure 34: Building mid-prices (HIGH+LOW)/2 and computing returns with `pct_change()`

| | Date_GBPEUR | High_GBPEUR | Low_GBPEUR | Date_SEKEUR |
|---|---|---|---|---|
| 4 | 2016-03-07 09:15:00 | 1.294 | 1.293 | 2016-03-07 09:15:00 |
| 5 | 2016-03-07 09:30:00 | 1.2943 | 1.2922 | 2016-03-07 09:30:00 |
| 6 | 2016-03-07 09:45:00 | 1.293 | 1.2913 | 2016-03-07 09:45:00 |
| 7 | 2016-03-07 10:00:00 | 1.2931 | 1.2921 | 2016-03-07 10:00:00 |
| 8 | 2016-03-07 10:15:00 | 1.2926 | 1.2921 | 2016-03-07 10:15:00 |
| ... | ... | ... | ... | ... |
| 12927 | 2016-09-07 17:00:00 | 1.1879 | 1.1867 | 2016-09-07 17:00:00 |
| 12928 | 2016-09-07 17:15:00 | 1.1883 | 1.1874 | 2016-09-07 17:15:00 |
| 12929 | 2016-09-07 17:30:00 | 1.188 | 1.1874 | 2016-09-07 17:30:00 |
| 12930 | 2016-09-07 17:45:00 | 1.1874 | 1.1866 | 2016-09-07 17:45:00 |
| 12931 | 2016-09-07 18:00:00 | 1.187 | 1.1869 | 2016-09-07 18:00:00 |

| | High_SEKEUR | Low_SEKEUR | Date_CADEUR | High_CADEUR | Low_CADEUR |
|---|---|---|---|---|---|
| 4 | 0.10728 | 0.10717 | 2016-03-07 09:15:00 | 0.6849 | 0.6841 |
| 5 | 0.10726 | 0.10719 | 2016-03-07 09:30:00 | 0.6844 | 0.6837 |
| 6 | 0.10728 | 0.10721 | 2016-03-07 09:45:00 | 0.6844 | 0.6839 |
| 7 | 0.10725 | 0.10719 | 2016-03-07 10:00:00 | 0.684 | 0.6835 |
| 8 | 0.10724 | 0.10718 | 2016-03-07 10:15:00 | 0.6839 | 0.6836 |
| ... | ... | ... | ... | ... | ... |
| 12927 | 0.10536 | 0.10531 | 2016-09-07 17:00:00 | 0.6897 | 0.6893 |
| 12928 | 0.10537 | 0.10534 | 2016-09-07 17:15:00 | 0.6902 | 0.6895 |
| 12929 | 0.10538 | 0.10536 | 2016-09-07 17:30:00 | 0.6902 | 0.6898 |
| 12930 | 0.10537 | 0.10536 | 2016-09-07 17:45:00 | 0.6902 | 0.6901 |
| 12931 | 0.10537 | 0.10537 | 2016-09-07 18:00:00 | 0.6901 | 0.6901 |

| | Avg_GBPEUR | Avg_SEKEUR | Avg_CADEUR | Ret_GBPEUR | Ret_SEKEUR |
|---|---|---|---|---|---|
| 4 | 1.29350 | 0.107225 | 0.68450 | 0.000812 | 0.000000e+00 |
| 5 | 1.29325 | 0.107225 | 0.68405 | -0.000193 | -1.110223e-16 |
| 6 | 1.29215 | 0.107245 | 0.68415 | -0.000851 | 1.865237e-04 |
| 7 | 1.29260 | 0.107220 | 0.68375 | 0.000348 | -2.331111e-04 |
| 8 | 1.29235 | 0.107210 | 0.68375 | -0.000193 | -9.326618e-05 |
| ... | ... | ... | ... | ... | ... |
| 12927 | 1.18730 | 0.105335 | 0.68950 | -0.000589 | 1.424231e-04 |
| 12928 | 1.18785 | 0.105355 | 0.68985 | 0.000463 | 1.898704e-04 |
| 12929 | 1.18770 | 0.105370 | 0.69000 | -0.000126 | 1.423758e-04 |
| 12930 | 1.18700 | 0.105365 | 0.69015 | -0.000589 | -4.745184e-05 |
| 12931 | 1.18695 | 0.105370 | 0.69010 | -0.000042 | 4.745409e-05 |

| | Ret_CADEUR |
|---|---|
| 4 | 1.389803e-03 |
| 5 | -6.574142e-04 |
| 6 | 1.461881e-04 |
| 7 | -5.846671e-04 |
| 8 | -1.110223e-16 |
| ... | ... |
| 12927 | 3.627131e-04 |
| 12928 | 5.076142e-04 |
| 12929 | 2.174386e-04 |
| 12930 | 2.173913e-04 |
| 12931 | -7.244802e-05 |

[12928 rows x 15 columns]

Figure 35: Final cleaned dataset

At this point, preprocessing is done. We now have clean FX return series at the smallest available time resolution, so we can move to the rest of Question E.

## II. Multiresolution correlations using Haar wavelets (Question E.a)

In this part, we look at how the three FX rates move together depending on the time scale. Using Haar wavelets, we compute multiresolution correlations between GBPEUR, SEKEUR and CADEUR, and we discuss whether an Epps effect appears.

The idea is to separate very short-term moves from smoother, longer-term moves. Then we compute correlations scale by scale (level by level), instead of using only one single correlation.

**Course reference:** we use the multiresolution framework based on the Haar wavelet transform (see lecture notes, page 291).

## Multiresolution analysis

The Haar wavelet allows an intuitive understanding of wavelets:

- It is a Daubechies wavelet with a single zero moment, so very rudimentary to approximate regular functions.
- The scaling function is $\phi = \mathbf{1}_{[0,1)}$ and the wavelet:

$$\psi(t) = \begin{cases} -1 & \text{if } 0 \leq t < 1/2 \\ 1 & \text{if } 1/2 \leq t < 1 \\ 0 & \text{otherwise.} \end{cases}$$

- The decomposition on $\phi$ and $\psi$ of a function $a\mathbf{1}_{[0,1/2]} + b\mathbf{1}_{[1/2,1]}$ allows us to understand multiresolution analysis.
- The approximation of daily returns $r_t$ at the $2^j$ scale, i.e. $\sum_{k\in\mathbb{Z}} \langle r, \phi_{j,k}\rangle \phi_{j,k}(t)$, is therefore based on returns of horizon $2^j$ calculated each day, with overlap.
- The detail is easily calculated on a date as a difference in returns. It is zero if the returns follow a trend, non-zero if they oscillate around an average.

Figure 36: Course slide (page 291): Haar wavelet and multiresolution analysis idea.

Before computing correlations, we first check that the three FX series are aligned. This is important: correlations only make sense if we compare the same timestamps.

```
[72]: # beefore we computing correlations, we quickly check that the three FX series are aligned
      # on the same timestamps (row by row). If not, we would need to merge on a common time index.
      aligned_gbp_sek = (df_clean["Date_GBPEUR"].values == df_clean["Date_SEKEUR"].values).all()
      aligned_gbp_cad = (df_clean["Date_GBPEUR"].values == df_clean["Date_CADEUR"].values).all()

      print("GBPEUR vs SEKEUR aligned:", aligned_gbp_sek)
      print("GBPEUR vs CADEUR aligned:", aligned_gbp_cad)

      GBPEUR vs SEKEUR aligned: True
      GBPEUR vs CADEUR aligned: True
```

Figure 37: Check that the three date columns are aligned (same timestamps).

We then take the three return series and drop missing values so they stay perfectly aligned: `Ret_GBPEUR`, `Ret_SEKEUR`, and `Ret_CADEUR`.

Now we apply a Haar multiresolution decomposition. The function builds **detail coefficients** at each level. At each level, we take pairs of points and compute: an average part (smooth move) and a detail part (local fluctuation). Then we keep going on the average part to go to a coarser scale.

After that, for each level, we compute correlations between the three FX pairs using the detail coefficients. To compare levels fairly, we standardize each level series(mean 0, std 1) before computing correlations.

[86]:
```python
# We take the clean 15min returns and drop NaN so the three series stay perfect aligned
returns = df_clean[["Ret_GBPEUR", "Ret_SEKEUR", "Ret_CADEUR"]].dropna(how="any")

def haar_wavelet_transform(x, levels=3):

    details_per_level = []
    current = x.copy()

    for _ in range(levels):
          # Haar works on pairs, so we just trim the last point if the length is odd
        current = current[: (len(current) // 2) * 2]

        # Haar step: average = approxmation,difference = detail
        approx = (current[::2] + current[1::2]) / 2
        detail = (current[::2] - current[1::2]) / 2

        details_per_level.append(detail)
        current = approx  # and we keep going on the smoother part

    return details_per_level

# 1 we compute the Haar detail coefficients for each FX return series
wavelet_results = {}
for col in returns.columns:
    wavelet_results[col] = haar_wavelet_transform(returns[col].values, levels=3)

# 2/ level by level, we compute correlations between the detail coefficients
correlations = {}
n_levels = len(wavelet_results["Ret_GBPEUR"])

for level in range(n_levels):
    gbp = wavelet_results["Ret_GBPEUR"][level]
    sek = wavelet_results["Ret_SEKEUR"][level]
    cad = wavelet_results["Ret_CADEUR"][level]
# We standardize (mean 0,std 1) so we compare levels fairly
    gbp = (gbp - gbp.mean()) / gbp.std()
    sek = (sek - sek.mean()) / sek.std()
    cad = (cad - cad.mean()) / cad.std()

    label = f"Level {level + 1}"
    correlations[label] = {
        "GBPEUR-SEKEUR": float(np.corrcoef(gbp, sek)[0, 1]),
        "GBPEUR-CADEUR": float(np.corrcoef(gbp, cad)[0, 1]),
        "SEKEUR-CADEUR": float(np.corrcoef(sek, cad)[0, 1]),
    }

# 3/Finally, we print the results in a clean  way
for lvl, corr in correlations.items():
    print(f"{lvl} correlations:")
    for pair, val in corr.items():
        print(f"  {pair}: {val:.4f}")
```

```
Level 1 correlations:
  GBPEUR-SEKEUR: 0.1601
  GBPEUR-CADEUR: 0.2182
  SEKEUR-CADEUR: 0.1742
Level 2 correlations:
  GBPEUR-SEKEUR: 0.1400
  GBPEUR-CADEUR: 0.3276
  SEKEUR-CADEUR: 0.2537
Level 3 correlations:
  GBPEUR-SEKEUR: 0.2997
  GBPEUR-CADEUR: 0.2260
  SEKEUR-CADEUR: 0.1600
```

Figure 38: Haar multiresolution decomposition and correlations by level (printed results).

We read the levels in time units. With 15-minute data, Level $j$ roughly corresponds to a horizon of about $(2^j) \times 15$ minutes: Level 1 $\approx$ 30 minutes, Level 2 $\approx$ 1 hour, Level 3 $\approx$ 2 hours. So higher levels mean a coarser time scale.

Here are the correlations we get:

**Level 1:** correlations are small overall: about 0.16 for GBPEUR-SEKEUR, about 0.17 for SEKEUR-CADEUR, and about 0.22 for GBPEUR-CADEUR. So at the shortest horizon, the three series do not move together that strongly.

**Level 2:** dependence becomes more visible for some pairs. GBPEUR-CADEUR increases to about 0.33 and SEKEUR-CADEUR rises to about 0.25, while GBPEUR-SEKEUR stays lower around 0.14.

**Level 3:** the ranking changes again. GBPEUR-SEKEUR becomes the most correlated pair at about 0.30, while GBPEUR-CADEUR goes back to about 0.23 and SEKEUR–CADEUR drops to about 0.16. So the pattern is not perfectly monotone across levels, which is normal: each level looks at a different time scale, not just a simple aggregation.

A simple way to interpret this is through the usual Epps effect. At very short horizons, correlations often look weaker. They become easier to see once we move to slightly longer time scales.

**Conclusion:** overall, correlations look weaker at the very highest frequency, and they become clearer when we move to coarser scales. We also see that the most correlated pair can change across levels, which shows that dependence is scale-dependent.

We now move to the Hurst exponent estimation to study the scaling behavior of volatility.

## III. Estimation of Hurst exponent and volatility scaling (Question E.b)

In this second part, we study the scaling behavior of the three FX return series. We estimate the Hurst exponent for GBPEUR, SEKEUR and CADEUR. Then we use these Hurst estimates to compute annualized volatilities in a way that does not rely on independence.

**Course reference:** the Hurst estimator used here is recalled in the lecture notes (pages 251-252).

## An estimate of $H$ for a known Gaussian volatility

We define empirical absolute moments on our sample $[0, T]$ with $1/N$ spacings:

$$M_k = \frac{1}{NT} \sum_{i=1}^{NT} |X(i/N) - X((i-1)/N)|^k. \tag{2}$$

$M_k$ is an estimate of $\mathbb{E}(|X(.) - X(. - 1/N)|^k)$, which value is:

$$\mathbb{E}(|X(.) - X(. - 1/N)|^k) = \frac{2^{k/2}\Gamma(\frac{k+1}{2})}{\Gamma(\frac{1}{2})} \sigma^k N^{-kH}. \tag{3}$$

It thus leads to the following estimate of $H$:

$$\check{H} = -\frac{\log(\sqrt{\pi} M_k / [2^{k/2}\Gamma(\frac{k+1}{2})\sigma^k])}{k \log(N)},$$

which converges towards $H$ at the rate $O((\sqrt{\varepsilon_N} \log(N))^{-1})$, when $N$ goes to infinity. Whatever $k$, it depends on $\sigma$.

Figure 39: Course slide (pages 251-252): idea of estimating $H$ from absolute moments.

From now $k = 2$ (estimator then stems from $\mathbb{E}\{(B_H(t) - B_H(s))^2\} = \sigma^2 |t - s|^{2H}$). We combine it with the same kind of statistic $M_2'$ based on observations in the same window but with halved resolution:

$$M_2' = \frac{2}{NT} \sum_{i=1}^{NT/2} |X(2i/N) - X(2(i-1)/N)|^2.$$

Then:

$$\hat{H} = \frac{1}{2} \log_2 \left( \frac{M_2'}{M_2} \right). \tag{4}$$

$\hat{H}$ converges almost surely toward $H$.

Figure 40: Course slide (pages 251-252): final estimator used here (with $k = 2$).

In practice, we implement the estimator with $k = 2$. We compute two empirical moments: one using step size 1 (all consecutive points), and one using step size 2 (every two points). Then we combine them to get an estimate of $H$.

Important detail: the estimator is defined for a price-like process $X_t$. Here we start from returns, so we build a price-like series by cumulating returns:

$$X_t = \sum_{i=1}^{t} r_i.$$

This gives a convenient proxy of a log-price path over the sample.

We now compute $H$ for the three FX series.

[84]:
```python
def Hurst_Exponent(X):
    k = 2  # we use the moment of order 2 (same estimator as in the slides)
    T = len(X)

    # quick safety check (we need enough points)
    if T < 4:
        return np.nan

    # 1/we compute M_2 with step size = 1
    M_2 = 0
    for i in range(1, T):
        M_2 += abs(X[i] - X[i - 1])**k
    M_2 /= (T - 1)

    # 2/ we compute M_2_prime with step size = 2
    M_2_prime = 0
    for i in range(1, T // 2):
        M_2_prime += abs(X[2 * i] - X[2 * i - 2])**k
    M_2_prime /= (T // 2 - 1)

    # 3/we apply H = 0.5 * log2(M_2_prime / M_2)
    return 0.5 * np.log2(M_2_prime / M_2)


# we take the 15-min returns from our cleaned dataset
ret_gbp = returns["Ret_GBPEUR"].values.astype(float)
ret_sek = returns["Ret_SEKEUR"].values.astype(float)
ret_cad = returns["Ret_CADEUR"].values.astype(float)

# we build X_t by cumulating returns (so it behaves like a price-like process)
X_GBPEUR = np.cumsum(ret_gbp)
X_SEKEUR = np.cumsum(ret_sek)
X_CADEUR = np.cumsum(ret_cad)

# we compute H for each FX rate
Hurst_GBPEUR = Hurst_Exponent(X_GBPEUR)
Hurst_SEKEUR = Hurst_Exponent(X_SEKEUR)
Hurst_CADEUR = Hurst_Exponent(X_CADEUR)

print("Hurst exponent for GBPEUR:", Hurst_GBPEUR)
print("Hurst exponent for SEKEUR:", Hurst_SEKEUR)
print("Hurst exponent for CADEUR:", Hurst_CADEUR)
```

```
Hurst exponent for GBPEUR: 0.686860136564295
Hurst exponent for SEKEUR: 0.6810236267055336
Hurst exponent for CADEUR: 0.6810811380086749
```

Figure 41: Hurst exponent estimation for the three FX series (implementation of the $k = 2$ estimator).

We obtain Hurst exponents all above 0.5: around 0.6869 for GBPEUR, 0.6810 for SEKEUR,

and 0.6811 for CADEUR. So the three values are close and suggest persistence compared to the Brownian benchmark ($H = 0.5$).

**Course reminder:** for a fractional Brownian motion interpretation: $H = 0.5$ corresponds to standard Brownian motion, $H > 0.5$ means persistent increments, and $H < 0.5$ means anti-persistent increments (lecture notes, page 210).

Increments of $B_H$ of duration $\tau$ are stationary: centred Gaussian of variance $\sigma^2|\tau|^{2H}$.

- if $H = 1/2$, the fBm is identical to a standard Brownian motion,
- if $H > 1/2$, the increments are persistent,
- if $H < 1/2$, the increments are anti-persistent.

Figure 42: Course slide (page 210): how to interpret $H$ (Brownian / persistent / anti-persistent).

**Notes:** we interpret these results with caution. This is intraday data, the sample is finite, and microstructure effects can affect scaling estimates. Also, we estimate $H$ on the cumulative sum of returns (a price-like proxy), not directly on log-prices.

### Annualized volatility using Hurst scaling

We now compute annualized volatilities using the Hurst scaling law, instead of the usual square-root-of-time rule.

**Course reference:** the scaling law is:

$$\sigma(\Delta t) \propto (\Delta t)^H,$$

so when we go from $\Delta t$ to $T$, we scale volatility like:

$$\sigma(T) = \sigma(\Delta t) \left(\frac{T}{\Delta t}\right)^H.$$

We use this because it does not assume independent returns (lecture notes, page 210).

Here the base step is 15 minutes. To annualize, we use the factor:

$$N = 252 \times 10 \times 4,$$

because we take 252 trading days per year, about 10 trading hours per day, and 4 points per hour (15-minute data). So:

$$\sigma_{\text{annual}} = \sigma_{15\text{min}}\, N^H.$$

We compute $\sigma_{15\text{min}}$ as the standard deviation of 15-minute returns on the sample, then we apply the scaling with the estimated $H$.

[106]:
```python
def calculate_annualized_volatility(returns_series, hurst_exponent):
    # first we just take the average return on our sample
    mean_return = sum(returns_series) / len(returns_series)

    # then we compute the volatility at the 15-min step (simple std formula)
    std_return = np.sqrt(
        sum((x - mean_return) ** 2 for x in returns_series) / (len(returns_series)))

    # now we scale it up to one year using the Hurst exponent (so we don't rely on independence)
    # for our dataset, we take 252 trading days per year and about 10 trading hours per day
    # with 15-min data, that's 4 points per hour
    return std_return * (252 * 10 * 4) ** hurst_exponent


#we apply the same formule to each FX returns series
annualy_vol_GBPEUR = calculate_annualized_volatility(
    returns["Ret_GBPEUR"].dropna().values.astype(float),
    Hurst_GBPEUR)
print(f"Annualized volatility for GBPEUR: {annualy_vol_GBPEUR:.6f}")

annualy_vol_SEKEUR = calculate_annualized_volatility(
    returns["Ret_SEKEUR"].dropna().values.astype(float),
    Hurst_SEKEUR)
print(f"Annualized volatility for SEKEUR: {annualy_vol_SEKEUR:.6f}")

annualy_vol_CADEUR = calculate_annualized_volatility(
    returns["Ret_CADEUR"].dropna().values.astype(float),
    Hurst_CADEUR)
print(f"Annualized volatility for CADEUR: {annualy_vol_CADEUR:.6f}")
```

```
Annualized volatility for GBPEUR: 0.350331
Annualized volatility for SEKEUR: 0.174243
Annualized volatility for CADEUR: 0.269808
```

Figure 43: Annualized volatility using Hurst-based scaling (15-minute step scaled to one year).

We obtain annualized volatilities of about: 0.3503 for GBPEUR, 0.1742 for SEKEUR, and 0.2698 for CADEUR.

**Conclusion (E.b):** we find similar Hurst exponents for the three FX series, all above 0.5. This suggests some persistence, but we keep in mind the limits of intraday data and the estimation method. Using $H$-based scaling then allows us to annualize volatility beyond the standard square-root-of-time rule.

**Final remark:** overall, these results show that FX behavior depends on the time scale, so time scale should be taken into account when analyzing dependence and risk.

# General conclusion (key results)

| Question | Main results |
|---|---|
| A (VaR) | Empirical 1-day VaR ($\alpha = 0.05$): VaR $\approx -0.037779 \Rightarrow \approx$ 3,777.91 euros (portfolio 100,000). |
| A(KDE VaR) | Kernel VaR (biweight KDE): VaR $\approx -0.034786 \Rightarrow \approx$ 3,478.61 euros; out-of-sample exceedances (2017–2018): $9/510 \approx 1.76\%$. |
| B (ES) | Expected Shortfall ($\alpha = 0.05$): ES $\approx -0.05004$. |
| C (EVT) | Pickands tail index: $\xi_{\text{gains}} \approx 0.5772$ (heavy tail), $\xi_{\text{losses}} \approx -0.5090$ (lighter tail); EVT VaR (losses): 90% $\approx 0.03141$, 95% $\approx 0.04354$, 99% $\approx 0.05958$. |
| D (Impact) | Bouchaud model estimates: $r \approx 0.7$, $\gamma \approx 2.0$, $V \approx 0.001992$, MSE $\approx 0.005377$. |
| E (Wavelets) | Haar multiresolution correlations: weaker at the finest scale, clearer at coarser scales (Epps-type effect). |
| E (Hurst) | Hurst: $H_{\text{GBPEUR}} \approx 0.6869$, $H_{\text{SEKEUR}} \approx 0.6810$, $H_{\text{CADEUR}} \approx 0.6811$; Annualized vol (H-scaling): GBPEUR $\approx 0.3503$, SEKEUR $\approx 0.1742$, CADEUR $\approx 0.2698$. |